

Министерство образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

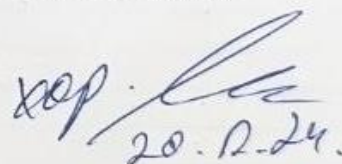
ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по курсу «Логика и основы алгоритмизации

в инженерных задачах»

на тему «Реализация поиска независимых множеств вершин
графа»


20.12.24.

Выполнил:

Студент группы 23BBB3 Кучин А.В.

Принял:

Митрохин М.А.

Пенза 2024

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет Вычислительной техники

Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ М.А. Широкин

« » 20

ЗАДАНИЕ

на курсовое проектирование по курсу

Логика и основы алгоритмизации в инженерных задачах
Студенту Кучину Алексею Викторовичу Группа 23 ВВВЗ
Тема проекта Реализация алгоритма поиска независимых множеств
Вершин графа

Исходные данные (технические требования) на проектирование

Разработка алгоритмов и программного обеспечения в соответствии
с данными заданием курсового проекта
Техническая записка должна содержать:

1. постановка задачи
2. теоретическая часть задания
3. описание алгоритма поставленной задачи
4. пример ручного расчета задачи и выводов
5. описание работы программы
6. тесты
7. список литературы
8. листинг
9. результат работы программы

Объем работы по курсу

1. Расчетная часть

Ручной расчет работы алгоритма

2. Графическая часть

Схема алгоритма в формате Блок-Схем

3. Экспериментальная часть

Тестирующие программы

Результаты работы программ на тестовых данных

Срок выполнения проекта по разделам

1 исследование теоретической части проекта

2 разработка алгоритма программы

3 разработка программы

4 тестирование и разработка программы

5 оформление пояснительной записки

6

7

8

Дата выдачи задания "12" 09.2024 г.

Дата защиты проекта " " "

Руководитель

Задание получил "12" 09

2024 г.

Студент Кутин А.В.

Содержание

Реферат	5
Введение	6
1. Постановка задачи	8
2. Теоретическая часть задания	9
3. Описание алгоритма программы	10
4. Описание программы	10
5. Тестирование	19
6. Ручной расчет задачи.....	23
Заключение.....	26
Список литературы	27
Приложение А.	28

Реферат

Отчет 31 стр, 14 рисунков.

НЕЗАВИСИМЫЕ МНОЖЕСТВА ВЕРШИН ГРАФА.

Цель исследования – разработка программы, способной выполнять алгоритм поиска независимых множеств вершин графа.

В работе рассмотрен способ рекурсивного поиска независимых вершин графа. Программа генерирует граф случайным образом и заносит его в файл, или, на выбор пользователя, использует заранее заполненный файл. Затем происходит поиск всех множеств независимых вершин используя рекурсию и записывает результат в указанный пользователем файл.

Введение

Данная курсовая работа посвящена разработке и реализации алгоритма поиска независимых множеств вершин в неориентированных графах. Теория графов, изучающая свойства графов – математических структур, представляющих собой множество вершин (узлов) и множество ребер (связей между вершинами), – является мощным инструментом для моделирования и анализа самых разнообразных объектов и процессов. Графы используются для представления сетей коммуникаций, социальных связей, молекулярных структур, транспортных систем и многих других объектов, где важны отношения между компонентами.

Независимые множества являются одним из ключевых понятий в теории графов. Независимое множество вершин в неориентированном графе – это такое подмножество вершин, что никакие две вершины из этого подмножества не соединены ребром. Проще говоря, это набор вершин, которые не имеют прямых связей между собой. Поиск независимых множеств имеет важное значение в решении многих задач оптимизации.

Например, в задаче о независимых заданиях, где каждое задание представляет вершину, а ребро между двумя вершинами указывает на конфликт между заданиями, независимое множество соответствует максимальному количеству заданий, которые могут быть выполнены одновременно. Другие примеры применения включают задачи покрытия, кодирования, расстановки частот в беспроводных сетях, а также задачи в биоинформатике, такие как поиск неперекрывающихся подструктур в молекулярных структурах.

Целью данной курсовой работы является разработка программного обеспечения реализующего алгоритм поиска всех независимых множеств вершин в неориентированном графе, представленном в виде матрицы смежности. Программа будет обеспечивать два основных режима работы: чтение графа из файла и генерацию случайного графа. Программа выведет найденные независимые множества в консоль и сохранит их в выходном файле. Работа также включает в себя обзор существующих методов поиска независимых множеств и анализ сложности разработанного алгоритма

1. Постановка задачи

Исходный граф в программе должен задаваться матрицей смежности, причём при генерации данных должны быть предусмотрены ограниченные условия.

Программа должна работать как со случайной генерацией графов, так и считывать графы из файла. Пользователь должен иметь выбор какой из этих вариантов использовать. При выборе случайной генерации пользователь должен указать размер графа. Так же в случае случайной генерации результат генерации должен сохраняться в текстовый файл, указываемый пользователем. Результат работы программы так же сохраняется в указанный пользователем файл.

Устройство ввода – клавиатура и мышь.

2. Теоретическая часть задания

В неориентированном графе $G = (V, E)$, где V – множество вершин, а E – множество ребер, независимое множество (также называемое стабильным множеством) – это подмножество вершин $I \subseteq V$ такое, что никакие две вершины в I не соединены ребром. Другими словами, для любых двух вершин, $v \in I$, $(u, v) \notin E$.

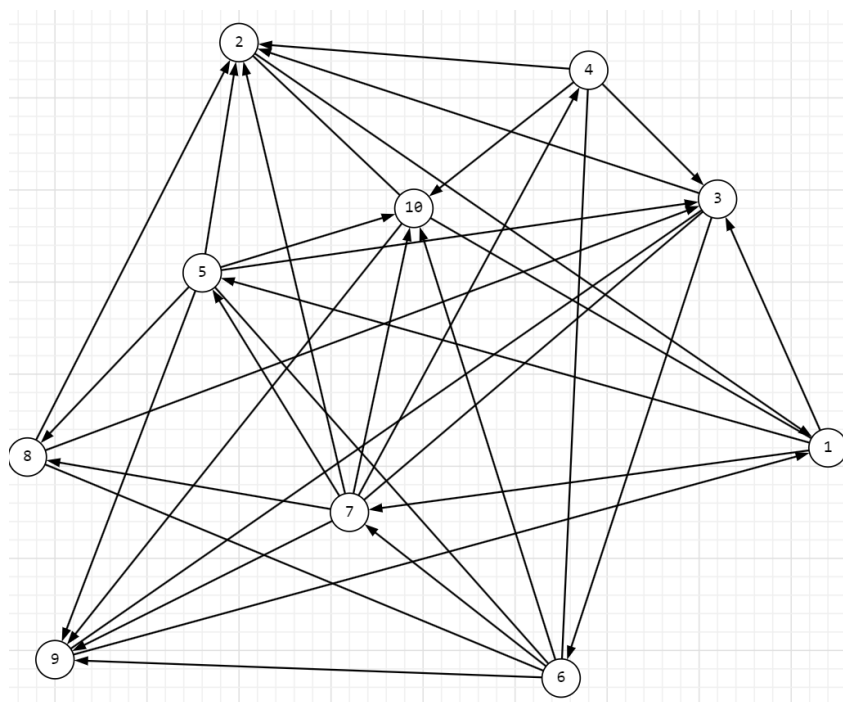


Рисунок 1– Неориентированный граф

Рассмотрим граф с вершинами $\{A, B, C, D, E\}$ и ребрами $\{(A, B), (B, C), (C, D), (D, E), (E, A)\}$, в котором:

$\{B, D\}$ – это независимое множество.

$\{A, C\}$ – это не независимое множество, так как A и C соединены не прямой, а через B .

$\{A, B, C\}$ - это не независимое множество.

$\{A, C, E\}$ - это не независимое множество

3. Описание алгоритма программы

Для реализации алгоритма поиска независимых вершин графа необходим исходный граф. Его можно как задать с помощью случайной генерации, указав его размеры в программе, так и считать с текстового файла, указанного пользователем.

Главной функцией, выполняющей задание является `findIndependentSets`, которая, используя рекурсию, проверяет все возможные множества на независимость, считая их независимыми до того момента, пока не будет доказано обратное. После проверки множества, в случае если оно действительно независимо, множество заносится в указанный пользователем ранее текстовый файл, а так-же выводится в консоль.

Ниже представлен псевдокод функции `findIndependentSets`:

ЕСЛИ `index` равно `numVertices` ТО:

`independent` = ИСТИНА

ДЛЯ `i` от 0 ДО `count - 1`:

ДЛЯ `j` от `i + 1` ДО `count - 1`:

ЕСЛИ `graph[subset[i]][subset[j]]` равно 1 ТО:

`independent` = ЛОЖЬ

ПЕРЕЙТИ к следующей итерации внешнего цикла

КОНЕЦ ДЛЯ

КОНЕЦ ДЛЯ

ЕСЛИ `independent` равно ИСТИНА ТО:

ВЫВЕСТИ `subset` на консоль

ЗАПИСАТЬ `subset` в `outputFile`

КОНЕЦ ЕСЛИ

ВЕРНУТЬ

КОНЕЦ ЕСЛИ

Рекурсивный вызов с включением вершины

Рекурсивный вызов без включения вершины

КОНЕЦ функции

Полный код программы можно увидеть в Приложении А.

4. Описание программы

Для написания данной программы использован язык программирования C++. Язык программирования C++ - универсальный язык программирования, который завоевал особую популярность у программистов, благодаря сочетанию возможностей языков программирования высокого и низкого уровней.

Проект был создан в виде консольного приложения Win32 (Visual C++).

Данная программа является многомодульной, поскольку состоит из нескольких функций: `readGraphFromFile`, `generateRandomGraph`, `findIndependentSets`,

Работа программы начинается с запроса действий пользователя. Если пользователь выбрал «Считать граф из файла», то у него попросят указать путь к файлу, в котором находится его граф. Затем его попросят указать файл, в который будет сохранён результат работы программы. В случае выбора пункта «Сгенерировать случайный граф», пользователя попросят ввести размер графа, путь с файлу для сохранения графа, а так-же путь к файлу для сохранения результата работы программы.

Общее представление программы на диаграмме модулей:

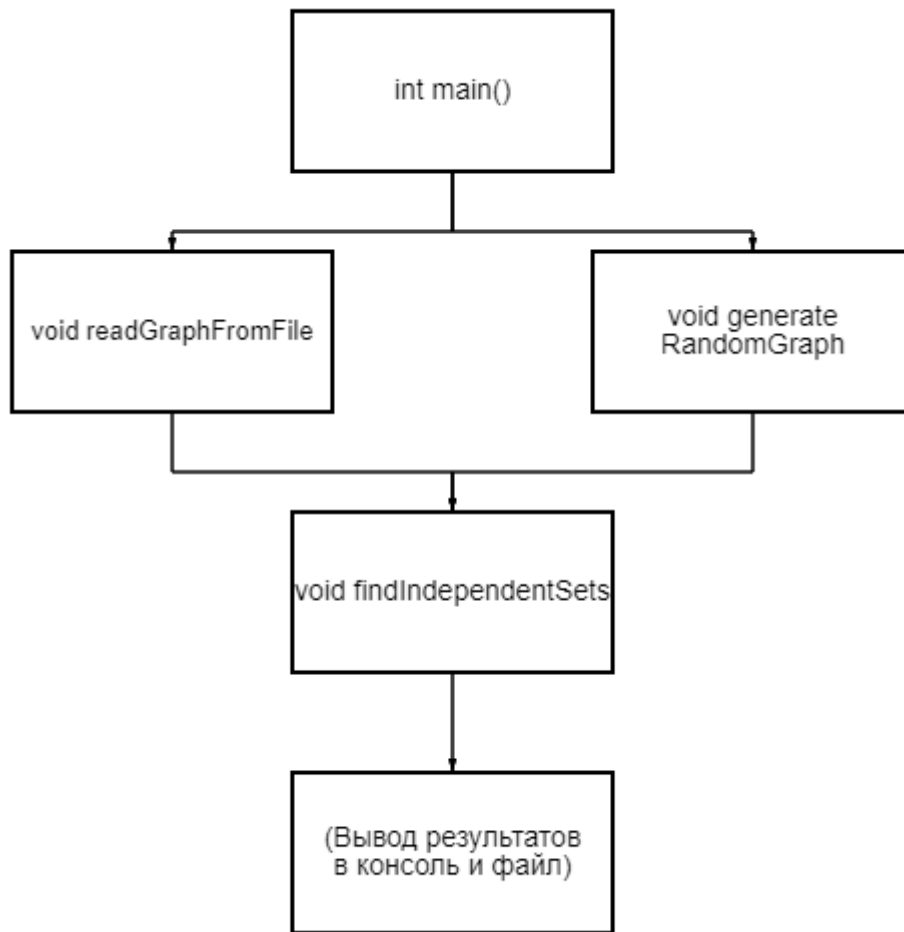


Рисунок 2 – Диаграмма модулей

Часть кода, ответственная за выбор функций:

```
printf("Выберите режим работы:\n");  
printf("1. Считать граф из файла\n");  
printf("2. Сгенерировать случайный граф\n");  
scanf("%d", &choice);
```

Эта часть кода просит пользователя выбрать режим работы:

- 1) Считывание графа из файла
- 2) Генерация случайного графа

Выбор пользователя сохраняется в переменную `choice`, используя которую с функциями `if` и `else if` используется необходимая часть кода.

Часть кода, ответственная за режим работы с чтением графа из файла:

```
if (choice == 1) {
    printf("Введите имя файла: ");
    scanf("%s", filename);
    std::ifstream inputFile(filename);
    if (!inputFile.is_open()) {
        std::cerr << "Ошибка открытия файла!" << std::endl;
        return 1;
    }
    inputFile >> numVertices;
    for (int i = 0; i < numVertices; ++i) {
        for (int j = 0; j < numVertices; ++j) {
            inputFile >> graph[i][j];
        }
    }
    inputFile.close();
}
```

Эта часть кода приводится в действие в том случае, если пользователь подал на ввод единицу, что соответствует режиму работы программы с чтением графа из файла. В ней пользователя просят ввести название файла, где в случае успеха происходит запись количества вершин в переменную numVertices и запись самого графа в двумерный массив graph[i][j]. В случае ошибки при открытии файла выводится соответствующее сообщение пользователю и происходит завершение работы программы

Часть кода, ответственная за режим работы со случайно сгенерированным графом:

```
else if (choice == 2) {
    printf("Введите количество вершин: ");
    scanf("%d", &numVertices);
    printf("Введите имя файла для сохранения графа: ");
    scanf("%s", graphFilename);
    graphFile.open(graphFilename);
    if (!graphFile.is_open()) {
        std::cerr << "Ошибка! Не удалось открыть файл для записи графа: " <<
graphFilename << std::endl;
        return 1;
    }
    generateRandomGraph(numVertices, graphFile);
}
```

```
graphFile.close();
}
```

Эта часть кода приводится в действие в том случае, если пользователь подал на ввод двойку, что соответствует режиму работы программы со случайной генерацией графа. В ней пользователя просят ввести количество вершин генерируемого графа, сохраняемое в переменную numVertices, и название файла в который будет записан сгенерированный граф, сохраняемое в переменную graphFilename. В случае если файл указан неверно, пользователю выводится сообщение об ошибке и происходит закрытие программы

Часть кода, отвечающая за обработку неверно выбранного режима работы программы

```
else {
    std::cerr << "Неверный выбор режима работы." << std::endl;
    return 1;
}
```

Эта часть кода приводится в действие в том случае, если пользователь подал на вход значение отличное от единицы или двойки. В этом случае выводится сообщение об ошибке и происходит закрытие программы.

Часть кода, ответственная за запись результата работы программы в файл.

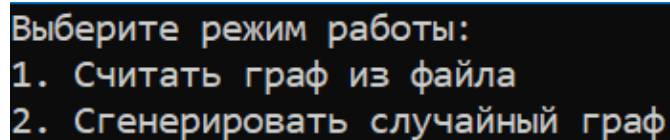
```
printf("Введите имя файла для вывода независимых множеств: ");
scanf("%s", outputFilename);

outputFile.open(outputFilename);
if (!outputFile.is_open()) {
    std::cerr << "Ошибка! Не удалось открыть файл для записи независимых
множеств: " << outputFilename << std::endl;
    return 1;
}
printf("Независимые множества:\n");
findIndependentSets(subset, 0, 0, outputFile);
outputFile.close();
```

Эта часть кода приводится в действие в том случае, если в частях ответственных за генерацию графа и его записи в файл или же считывании графа с пользовательского файла отсутствуют ошибки. В этом случае приводится в работы рекурсивный алгоритм поиска множеств независимых вершин графа, так-же называемый «жадный». После работы алгоритма пользователя просят ввести название файла, сохраняемое в переменную `outputFilename`, в который будет сохранен результат работы. В случае корректного ввода происходит запись результата в файл и завершение программы, в ином случае выводится сообщение об ошибке и происходит завершение работы программы

5. Демонстрация работы программы.

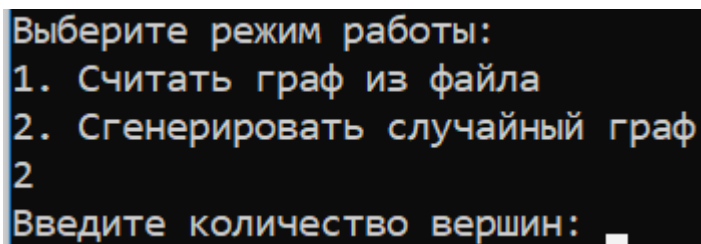
Ниже можно увидеть оформление начального запроса и дальнейшие действия с ним.



```
Выберите режим работы:  
1. Считать граф из файла  
2. Сгенерировать случайный граф
```

Рисунок 3 – Стартовое окно программы

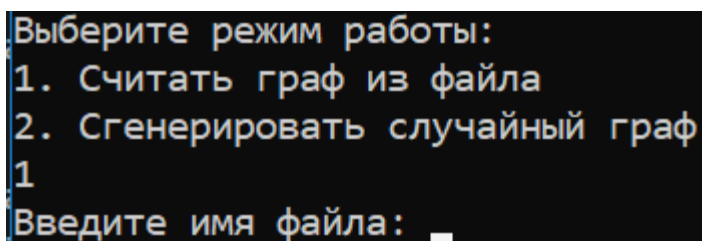
Выбор пункта со случайной генерацией графа и запрос на ввод количества вершин



```
Выберите режим работы:  
1. Считать граф из файла  
2. Сгенерировать случайный граф  
2  
Введите количество вершин: _
```

Рисунок 4 – Случайная генерация графа

Выбор пункта со считыванием графа из файла с последующим запросом файла от программы



```
Выберите режим работы:  
1. Считать граф из файла  
2. Сгенерировать случайный граф  
1  
Введите имя файла: _
```

Рисунок 5 – Считывание из файла

Выбор файла для записи результата работы программы, а так-же вывод результата в консоль.

```
Выберите режим работы:
1. Считать граф из файла
2. Сгенерировать случайный граф
2
Введите количество вершин: 3
Введите имя файла для сохранения графа: C:\Users\Admin\Desktop\try.txt
Введите имя файла для вывода независимых множеств: C:\Users\Admin\Desktop\check.txt
Независимые множества:
0 1
0
1 2
1
2
```

Рисунок 6 – Поиск независимых множеств

6. Тестирование

Среда разработки Microsoft Visual Studio 2022 предоставляет все средства, необходимые при разработке и отладке многомодульной программы.

Тестирование проводилось в рабочем порядке, в процессе разработки и после завершения написания программы. В ходе тестирования было выявлено и исправлено множество проблем, связанных с вводом и обращением к данным, изменением формата используемых данных, алгоритмом программы, взаимодействием функций.

Ниже продемонстрирован результат тестирования программы при вводе пользователем файла с исходным графом, ввода файла для сохранения результата алгоритма, ошибки при неправильном указании файла.

Таблица 1 – Описание ожидаемого поведения программы.

Описание теста	Предусловие	Последовательность	Ожидаемый результат
Запуск программы	ОС загружена, открыт каталог с программой в которой лежит код exe файл	Запустить ailab1.exe	Вывод пользователю списка с выбором режима работы программы
Выбор режима работы с пользовательским графом в файле	Программа запущена, выведено меню пользователя	Выбрать режим работы считывания графа из файла	Вывод пользователю сообщения с просьбой задать файл для чтения
Задание верного пути к файлу для чтения	Программа запущена, выведена просьба ввести файл для чтения	Ввести полный путь к файлу, содержащему исходный граф	Вывод пользователю сообщения с просьбой ввода файла для записи результата программы

Продолжение таблицы 1:

Описание теста	Предусловие	Последовательность	Ожидаемый результат
Задание верного пути к файлу для записи	Программа запущена, выведена просьба ввести файл для записи	Ввести полный путь к файлу для записи результата программы	Вывод пользователю результат работы программы в консоль, запись результатов работы программы в указанный файл
Задание неверного пути к файлу для чтения	Программа запущена, выведена просьба ввести файл для чтения	Ввести неверный путь к файлу, содержащему граф	Вывод сообщение об ошибке чтения данных из файла
Задание неверного пути к файлу для записи	Программа запущена, выведена просьба ввести файл для записи	Ввести неверный путь к файлу, предназначенному для записи результата работы программы	Вывод сообщение об ошибке записи данных в файл
Выбор режима работы со случайной генерацией графа	Программа запущена, выведено меню пользователя	Выбрать режим работы случайной генерации графа	Вывод пользователю сообщения с просьбой ввести файл для записи графа
Ввод размера генерируемого графа	Программа запущена, выведена просьба ввести размер генерируемого графа	Ввести целое положительное число	Вывод пользователю сообщения с просьбой ввести файл для записи случайно сгенерированного графа

Продолжение таблицы 1:

Описание теста	Предусловие	Последовательность	Ожидаемый результат
Ввод пути к файлу для записи случайно сгенерированного графа	Программа запущена, выведена просьба ввести файл для записи	Ввести полный путь к файлу для записи случайно сгенерированного графа	Вывод пользователю сообщения с просьбой ввести файл для записи результатов работы программы

Тестирование на ввод файла для чтения:

```

Выберите режим работы:
1. Считать граф из файла
2. Сгенерировать случайный граф
1
Введите имя файла: C:\Users\Admin\Desktop\try.txt
Введите имя файла для вывода независимых множеств:

```

Рисунок 7 – Тестирование при вводе файла для чтения.

Тестирование на ввод файла для сохранения:

```

Выберите режим работы:
1. Считать граф из файла
2. Сгенерировать случайный граф
1
Введите имя файла: C:\Users\Admin\Desktop\try.txt
Введите имя файла для вывода независимых множеств: C:\Users\Admin\Desktop\check.txt
Независимые множества:
0 1
0
1
2

```

Рисунок 8 – Тестирование при вводе файла для сохранения.

Тестирование на ввод заведомо неверного пути к файлу:

```
Выберите режим работы:  
1. Считать граф из файла  
2. Сгенерировать случайный граф  
1  
Введите имя файла: C:\Users\Admin\Desktop\neverno.txt  
Ошибка открытия файла!
```

Рисунок 9 – Тестирование при вводе заведомо неверного пути к файлу.

Проверка записи в файл случайной генерации графа, с указанием количества вершин:

```
try.txt – Блокнот  
Файл Правка Форм  
3  
0 0 1  
0 0 1  
1 0 0
```

Рисунок 10 – Запись случайной генерации графа в файл.

Проверка записи в файл результатов работы программы

```
check.txt – Блокнот  
Файл Правка Формат  
0 1  
0  
1  
2
```

Рисунок 11 – Запись результатов работы программы в файл.

Таблица 2 – Поведение программы при тестировании.

Описание теста	Ожидаемый результат	Полученный результат
Выбор чтения графа с файла	Вывод пользователю предложения указать путь к файлу с графом	Верно
Выбор случайной генерации графа	Вывод пользователю предложение указать количество вершин графа	Верно
Ввод пути к файлу с пользовательским графом	Успешное открытие файла и дальнейшая обработка данных	Верно
Ввод заведомо ложного пути к файлу	Вывод сообщение об ошибке чтения файла	Верно
Ввод пути к файлу для записи результата работы программы	Успешное открытие файла и дальнейшая запись результатов работы	Верно

6. Ручной расчет задачи

Проведем проверку программы посредством ручных вычислений на примере графа с 3 вершинами (рисунок 12), сгенерированного автоматически в программе.

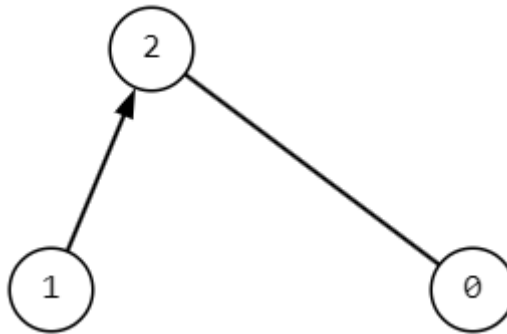


Рисунок 12 – Граф.

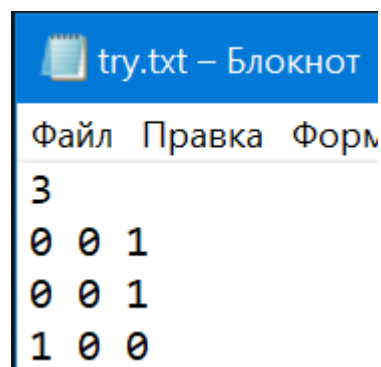


Рисунок 13 –Результат генерации.

Рассмотрим все множества и проверим, являются ли они независимыми:

Множество $\{0; 1\}$ не имеет общих ребер, оно может считаться независимым

Множество $\{0; 2\}$ имеет общее ребро, оно не может считаться независимым

Множество $\{1; 2\}$ имеет общее ребро, оно не может считаться независимым

Множество $\{0; 1; 2\}$ имеет общие ребра, оно не может считаться независимым

Каждая вершина по отдельности может считаться независимым множеством в том случае, если у нее нет петли.

Следовательно, результат ручных расчетов следующий:

{0}

{1}

{2}

{0; 1}

Ручной расчёт полностью совпадает с работой программы

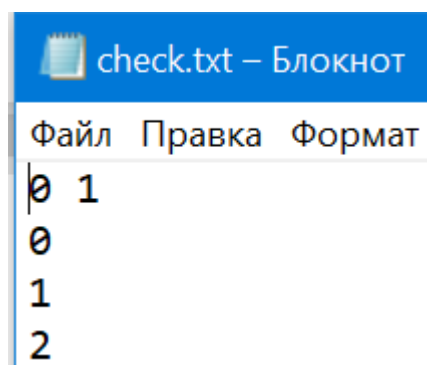


Рисунок 14 –Результат работы программы.

Заключение

Таким образом, в процессе создания данного проекта разработана программа, реализующая поиск всех независимых множеств графа в Microsoft Visual Studio 2022.

При выполнении данной курсовой работы были получены навыки разработки программ и освоены приемы создания двудольных графов, и динамических структур данных. Углублены знания языка программирования C++.

Недостатком разработанной программы является примитивный пользовательский интерфейс и высокая сложность алгоритма, что влечет за собой долгий просчёт для графов с большим количеством вершин.

Код написан максимально компактно и интерфейс удобен для использования. Полученные знания можно применять в дальнейшей деятельности.

Список литературы

1. Герберт Шилдт. С++. Базовый курс. 2014 г.
2. В. Л. Дольников О. П. Якимова. Основные алгоритмы на графах. 2011 г.
3. Уилсон Р. Введение в теорию графов. Пер. с англ. 1977. 208 с.

Приложение А.

Листинг программы.

```
#define _CRT_SECURE_NO_WARNINGS
#define MAX_VERTICES 100
#include <iostream>
#include <fstream>
#include <cstdlib>
#include <ctime>
#include <locale.h>
int graph[MAX_VERTICES][MAX_VERTICES];
int numVertices;

void readGraphFromFile(const char* filename) {
    std::ifstream inputFile(filename);
    if (!inputFile.is_open()) {
        std::cerr << "Ошибка открытия файла!" << std::endl;
        exit(1);
    }
    inputFile >> numVertices;
    for (int i = 0; i < numVertices; ++i) {
        for (int j = 0; j < numVertices; ++j) {
            inputFile >> graph[i][j];
        }
    }
    inputFile.close();
}

void generateRandomGraph(int numVertices, std::ofstream& graphFile) {
    srand(time(0));
    ::numVertices = numVertices;
    graphFile << numVertices << std::endl;
    for (int i = 0; i < numVertices; ++i) {
        for (int j = 0; j < numVertices; ++j) {
            if (i == j) {
                graph[i][j] = 0;
            }
            else {
                graph[i][j] = rand() % 2;
                graph[j][i] = graph[i][j];
            }
            graphFile << graph[i][j] << (j == numVertices - 1 ? "" : " ");
        }
    }
}
```

```

        graphFile << std::endl;
    }
}

void findIndependentSets(int subset[], int index, int count, std::ofstream&
outputFile) {
    if (index == numVertices) {
        bool independent = true;
        for (int i = 0; i < count; ++i) {
            for (int j = i + 1; j < count; ++j) {
                if (graph[subset[i]][subset[j]] == 1) {
                    independent = false;
                    break;
                }
            }
            if (!independent) break;
        }
        if (independent) {
            for (int i = 0; i < count; ++i) {
                printf("%d ", subset[i]);
            }
            printf("\n");

            for (int i = 0; i < count; ++i) {
                outputFile << subset[i] << (i == count - 1 ? "" : " ");
            }
            outputFile << std::endl;
        }
        return;
    }
    subset[count] = index;
    findIndependentSets(subset, index + 1, count + 1, outputFile);
    findIndependentSets(subset, index + 1, count, outputFile);
}

int main() {
    setlocale(LC_ALL, "RU");
    int choice;
    char filename[100], outputFilename[100], graphFilename[100];
    int subset[MAX_VERTICES];
    std::ofstream outputFile, graphFile;
    printf("Выберите режим работы:\n");
    printf("1. Считать граф из файла\n");

```

```

printf("2. Сгенерировать случайный граф\n");
scanf("%d", &choice);

if (choice == 1) {
    printf("Введите имя файла: ");
    scanf("%s", filename);
    std::ifstream inputFile(filename);
    if (!inputFile.is_open()) {
        std::cerr << "Ошибка открытия файла!" << std::endl;
        return 1;
    }
    inputFile >> numVertices;
    for (int i = 0; i < numVertices; ++i) {
        for (int j = 0; j < numVertices; ++j) {
            inputFile >> graph[i][j];
        }
    }
    inputFile.close();
}
else if (choice == 2) {
    printf("Введите количество вершин: ");
    scanf("%d", &numVertices);
    printf("Введите имя файла для сохранения графа: ");
    scanf("%s", graphFilename);
    graphFile.open(graphFilename);
    if (!graphFile.is_open()) {
        std::cerr << "Ошибка! Не удалось открыть файл для записи графа: "
<< graphFilename << std::endl;
        return 1;
    }
    generateRandomGraph(numVertices, graphFile);
    graphFile.close();
}
else {
    std::cerr << "Неверный выбор режима работы." << std::endl;
    return 1;
}

printf("Введите имя файла для вывода независимых множеств: ");
scanf("%s", outputFilename);

outputFile.open(outputFilename);

```

```

        if (!outputFile.is_open()) {
            std::cerr << "Ошибка! Не удалось открыть файл для записи независимых
множеств: " << outputFilename << std::endl;
            return 1;
        }
        printf("Независимые множества:\n");
        findIndependentSets(subset, 0, 0, outputFile);
        outputFile.close();
        return 0;
    }

```